

Multi-Region AI-Powered Banking Application

Engineering Mission-Critical, High-Availability Cloud Infrastructure with Zero-Downtime Tolerance

Architecture: Active-Active DR
Tools: Terraform & Github Actions

Author: Ikenna Anasieze - Cloud & AI Engineer
Repository: Github.com/donaldnaz/Financeapp

1. Overview

Modern banking systems operate under rigorous availability frameworks. Even brief structural disruptions can inflict significant commercial damage, invite regulatory penalties, and compromise consumer confidence. This project documents the end-to-end design, implementation, and orchestration of a resilient, AI-powered banking application deployed on Amazon Web Services (AWS).

The system features a multi-region deployment architecture designed to guarantee immediate failover recovery without human intervention, ensuring continuous availability during widespread cloud outages. Every infrastructure asset is declared as code, allowing rapid deployment, predictable testing environments, and automated decommissioning.

2. Problem to Solve

Traditional financial monolithic systems and naive single-region cloud architectures are vulnerable to systemic regional downtime, or regional cloud service disruptions. The design criteria for this system were framed around eliminating two primary vulnerabilities:

- Recovery Time Objective (RTO):** The maximum acceptable duration of downtime before service restoration. In mission-critical environments, the target metric must approach $RTO \rightarrow 0$.
- Recovery Point Objective (RPO):** The maximum acceptable age of data that can be lost due to an outage. For financial ledgers, transactional consistency requires an explicit target of $RPO = 0$.

3. Technology Stack

The stack incorporates specialized tools segmented into logical infrastructure layers to ensure structural rigidity and separation of concerns:

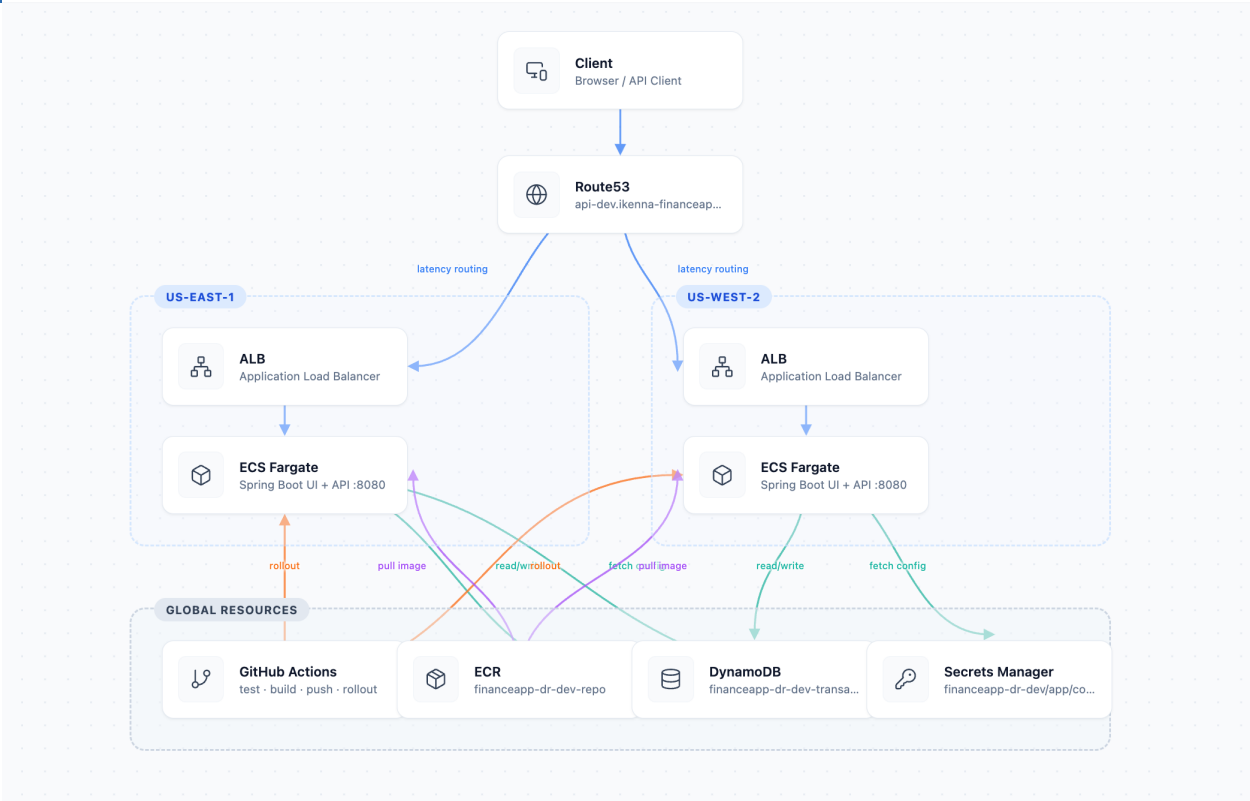
Infrastructure Layer	Technology Utilized	Functional Role
Cloud Provider	Amazon Web Services (AWS)	Global cloud infrastructure, cross-region networking, and managed abstractions.
Infrastructure as Code	Terraform	Declarative state files, modular network blocks, and multi-provider regional targeting.
CI/CD Engine	GitHub Actions	Automated build execution, application containerization, and structural testing

		pipelines.
Compute Tier	AWS ECS / Fargate	Serverless container execution, horizontal scaling, and isolated runtime layers.
Data Storage	Amazon DynamoDB	Global Tables implementation providing active-active cross-region multi-master synchronization.
Traffic Management	AWS ALB & Route 53	routing, continuous health profiling, and DNS failover record automation.
Artificial Intelligence	Cursor IDE	Natural Language processing for conversational banking, anomaly detection, and insights.

4. Architectural Flow

The architecture establishes isolation across two distinct geographic zones: us-east-1 (Primary Region) and us-west-2 (Secondary Region).

Client Request → Route 53 (Health-Monitored) → Regional ALB → ECS Fargate Microservices → DynamoDB Global Tables ↔ Multi-Region Cross-Replication.



5. Business Impact

Transitioning from a traditional cloud posture to this multi-region declarative framework yields clear business and operational advantages:

- **Uninterrupted Business Continuity:** The application achieves true regional disaster isolation. Total failure of an entire AWS data-center hub triggers automated recovery procedures without service interruption for users.
- **Optimized Cost Efficiency:** By utilizing AWS Fargate serverless compute units and on-demand DynamoDB pricing models, the operational maintenance costs for running this multi-region infrastructure baseline are minimized to approximately \$8.00 / day during idle testing.
- **Elimination of Manual Operations:** Infrastructure code guarantees environment parity. The exact topology can be reproduced instantly, eliminating manual execution variance or configuration drift during disaster scenarios.